Modèle de Génération Automatique de Code à partir de Maquettes GUI Haute Fidélité

Utilisation des techniques de Deep Learning

Résumé de l'article de Samir et al. (2024)

21 juin 2026

Introduction

- Les interfaces graphiques (GUI) sont essentielles pour l'interaction utilisateur-logiciel en raison de leur nature visuelle .
- Le flux de travail classique repose sur la création de **maquettes (mockups)** de haute fidélité avant l'implémentation.
- Le développement des GUI est souvent une tâche **manuelle et chronophage** .
- Environ 45% de la taille du code d'un logiciel concerne l'interface utilisateur, occupant près de 50% du temps de développement.

Le Problème

Constats majeurs

- Processus fastidieux et répétitif dû aux changements fréquents de spécifications .
- Les développeurs passent trop de temps sur l'UI au détriment de la logique métier du logiciel.
- Tâche sujette aux erreurs lors de l'extraction des éléments visuels et de leur hiérarchie .
- Nécessité de réimplémenter l'interface pour chaque plateforme native (Android, iOS, Web) .

Problématique

Comment automatiser efficacement la conversion d'une image de maquette haute fidélité en code frontal multiplateforme tout en conservant la hiérarchie visuelle ?

- Compréhension visuelle par la machine (Computer Vision) .
- Diversité des designs et complexité des composants GUI.
- Manque de précision des jeux de données existants (Rico, etc.).

Solutions apportées : Architecture

L'étude propose un pipeline en cinq phases :

1. **Préparation du modèle :** Utilisation de YOLOv7 avec réglage d'hyperparamètres (K-Mean++ pour les anchor boxes).
2. **Détection d'objets :** Séparation en deux modèles distincts pour améliorer la précision :
   - **Modèle Atomique :** (boutons, icônes, etc.) .
   - **Modèle Conteneur :** (barres d'outils, tiroirs, etc.) .
3. **Post-traitement :** Extraction des styles (couleurs, tailles) et du texte via OCR (Tesseract).
4. **Hiérarchie du layout :** Algorithme de groupement pour recréer la structure spatiale.
5. **Génération de code :** Production de code **Flutter** multiplateforme .

Solutions apportées : Pipeline de données

- **Nettoyage du dataset :** Création d'un pipeline de prétraitement pour corriger les erreurs du dataset Rico (éléments mal alignés, étiquettes incorrectes) .
- **Équilibrage :** Technique pour assurer une distribution uniforme des classes dans l'entraînement .
- **Résultats :** Précision moyenne (mAP) de **91,37%** pour les éléments atomiques et **87,40%** pour les conteneurs .

Limites

- **Taille des données :** Les jeux de données d'entraînement restent relativement petits malgré les bons résultats .
- **Cas complexes :** Difficultés persistantes pour certains agencements hiérarchiques spécifiques (ex : alignement complexe de cartes verticales).
- **Statisme :** Incapacité à distinguer certains types dynamiques comme la VIDEO de l'IMAGE sur des captures statiques.
- **Dépendance à la qualité :** Nécessite une vérification manuelle pour assurer un étiquetage complet lors de l'entraînement .

Conclusion

- Le modèle démontre que le **Deep Learning** est parfaitement adapté à la reconnaissance visuelle des composants GUI .
- L'approche à **deux modèles (YOLOv7)** surpasse les approches à modèle unique en raison de la diversité des échelles d'objets .
- Solution prometteuse pour réduire drastiquement le temps de développement front-end .
- **Futur :** Augmenter la diversité des données et affiner l'algorithme de groupement spatial .